Start Your Engines!

Game engines are behind all that's pretty in your favourite games. But how do they work?

Bhaskar Sarma

So how does a game run? Well, games have terrain maps, sound files, executables, etc., among other components. But what makes all these and more work together to give a seamless and immersive experience is the gaming engine. To understand the significance of the game engine, consider a car-without the engine—all you have is a big metal-and-plastic hulk. Similarly, without the game engine, a game is only a collection of files.

As today's games become more and more complicated, the capabilities of engines have increased. However, the functions of a game engine can be boiled down to a few basic areas which are explained below.

The Renderer

This is what transforms millions of lines of codes into eye candy that we see in the game. It is where the heaviest part of the CPU processing is involved. The renderer works actively with the hardware (graphics cards), drivers, APIs (Direct3D, OpenGL) to get those pixels to work their magic. Since the games are all 3D, there is a lot of high-level 3D mathematics involved in terms of angles, polygons and so on. In modern games polygons make up the game world—everything you see is made up of hundreds of thousands of vertices and surfaces. The sexy Lara Croft was all polygons, and so are the bad guys and the monsters that she fights. Magnify the game to its highest level, and what looked like smooth surfaces begin to look jagged. The renderer creates the view, depending on the position of the camera in the game world, which changes constantly as the character moves about. It has to load terrains every time the camera angle changes, and therefore performs on-the-fly—it won't create the objects in the next room until you enter it—ditto for those objects that are out of sight.

Physics And AI

Consider this scenario—you shoot a bad guy who is standing above you, and his body rises up to stick to the ceiling, like a helium filled dummy. That is not the way things happen in real world—which brings us to game physics. Realistic physics effects are very tough to simulate, and if done well, make the gameplay all the more interesting. In fact, games use specialised software called physics engines to handle effects like gravity, collision, inertia and acceleration. Physics engines like Havok and PhysX have been used in games like the *Halo* series, *Company of Heroes*, *Half-Life 2*, *BioShock* and *World in Conflict* to deliver real life effects.

Artificial Intelligence (AI) is another component of the game play that is very crucial. We're talking about non-playing characters—or bots—who are supposed to exhibit human like behaviour, like getting aggressive when a comrade is shot, taking cover and so on. The bots would not be able to see through walls, and what's more they won't walk through them. In fact, game world navigation by bots, which essentially means not walking through windows, travelling across corners and ignoring staircases is an important aspect of how advanced the game is.

Lighting And Textures

Games focus on realism, and without lighting effects there would be none. Imagine a world where light didn't reflect off smooth surfaces, and no shadows are thrown by objects. It's unreal, not to mention drab. Game engines handle the task of lighting and shadows using a combination of techniques. Sometimes, the polygons are lit differently depending on the placement of light source in the world. Again, this changes dynamically with the camera angle, and is what lets water bodies reflect light continuously, or allow for shadows. This is also memory intensive, as FPS games change camera angles quite fast and a fair amount of number crunching has to be done by the CPU.

Another area where light effects create realism is in texture. A game world has a variety of surfaces, some smooth and others rough. The technique used to display these textures is called bump mapping, and it differs from light effects in that the bumps don't change their positions with camera angles. Bump mapping provides depth to the images and creates variety in terrain—the surface of the earth in a flight simulator, for example.

Sound

Sound effects are one of the most important factors that make today's games the immersive experiences that they are. FPS games place a lot of emphasis on sound and it takes a fair bit of work before all the effects are aurally displayed. In most of the cases the sound that one hears in a game depends upon the position of the camera, and obstruction and occlusion play an important role here. When a conversation is taking place in another room the voices appear muted and muffled. This is occlusion, while obstruction is when the obstacle between the sound source and the player is something like a pillar. This difference is determined based on the calculation of presence of obstacles in the world, and can rapidly change. Sound effects can also be tweaked depending on the environment—for example in a corridor, underwater or in a cinema hall.

Network

Today's games are designed to be played over a network, and also incorporate online multiplayer options. Over a network, there are two models in which a game can be played—peer-to-peer and client-server. In peer-to-peer if a game is set up on four machines, each machine will play its own copy of the game while incorporating updates from the other machines—they all connect to each other. In the client-server model, all computers still run the game, but the server handles and decides the location of players, who killed who, etc.

Multiplayer games use UDP packets to exchange information between machines. Since there is always the risk of loss of packets across the network, there are methods to handle packet loss and let the players carry on. One method is client prediction where the game engine on the client side—in the absence of updates from the server—would predict what would happen depending on the input. Game engines reduce the size of packets by restricting players to a world view that is nearest to them, and also check and ban cheaters who employ unfair means to score hits.

Game Control

Game control is how you use inputs to control your game. This is a very important area and unless controls are intuitive and simple, the player won't be comfortable playing the game. The game engines take inputs from the input devices like keyboard, mouse and joysticks and transplant them into the game world. Good game engines also automate a lot of game control actions, like enabling the player to pick up weapons, ammo and medpacks without any clicks and keystrokes. The game engine controls and records all the camera positions in terms of coordinates in the game world which is then used for restarting saved games.

Weapon Systems

Different weapons are used in different ways in a game, and they kill in different ways. A sniper rifle kills instantly, while a rocket would take some time to travel. Again, weapons like grenades kill or hurt anyone within the blast radius .The game engine uses preset routines called traces, which checks each polygon in the trajectory of the projectile for collisions to determine if something has to be hit or not. It also has to look out for obstructions and cover—it would be pretty dumb if you got hit while hiding behind a wall. Then there are things like trajectory to consider: missiles and grenades usually fly in projectile motion, and how far they travel would depend on the angle at which the projectile is thrown. Simple number crunching, but when lots of guns are being fired and grenades lobbed the engine has to be pretty nimble.

Memory Management

Running a game is very memory intensive, and memory is an extremely limited resource, even with the oodles of memory which modern computers and graphics cards have. Textures and light effects are only a few of the tasks that hog memory, and the game engine needs a few aces up its sleeves if the game does not have to display as a slideshow. Many games have a lot of fog and smoke effects, which simulate real life situations. These fog effects are created by techniques called volumetric fogging, alpha testing and blending. Volumetric fogging creates the effect of going through a cloud, while another type of fogging depends on the distance of objects from the camera. This saves a lot of memory from having to store unnecessary texture maps.

Notice the realistic fog effect, along with sparkiling waters

Another approach to saving memory is the alpha testing and blending technique, along with depth testing. Game worlds are in 3D, with distances along the Z co-ordinate giving the appearance of depth. Depth testing and alpha blending works in almost the same ways—basically, testing the Z distance of the polygons and rendering those that are nearest on screen, and not rendering hidden pixels. This speeds up frame rates because pixels don't have to be drawn and redrawn too many times, especially useful in games like *Heretic II*, where characters threw multicoloured spells. Without depth testing this would have resulted in a pixel being drawn multiple number of times for the same frame.

This bazooka will hit its target, thanks to weapon trajectories

Game Over

Game engines are like SDKs, which offer a set of readymade tools that enable game creators and programmers to concentrate on the more creative aspects of the game instead of doing grunt work. They are built in such a way that multiple games can use the same game engine, much like different cars that are built around a single internal combustion engine. Game engines started off in *Doom* and have been in use since then in many ground breaking games. Most games are built around a few game engines like the *Doom* engine, *Unreal* engine, *Cube* engine and the *Quake* engine. Game engines can be both proprietary as well as open source (*Crystal Space*, *Open Dynamics Engine*). Though any type of game can be built around game engines, they have been extensively used with the FPS genre as this requires interaction at human scales.

Game engines are highly technical, and the ways in which they influence game elements can fill several tomes. Game engines are constantly being upgraded with each generation, and it does not take long for a feature to become mainstream from niche.■

bhaskar_sarma@thinkdigit.com